Programming Fundamentals

Week 5 – Dictionaries Revision

Programming Fundamentals Team

SETU

2026-10-16

Week 5: Let's Revisit Dictionaries

We didn't have our second live lecture last week, so we'll take a little time to revisit **dictionaries** before moving on.

Today we will practise:

- Creating and reading dictionaries
- Adding and changing values
- Checking for keys
- Looping through dictionaries
- Using dictionaries with lists and functions

> The aim is to understand what the code is doing, not to memorise every method.

A Quick Reminder: Lists vs Dictionaries

A **list** stores items in an order. We normally access them using an index.

```
dog = ["Buddy", "Labrador", 4]
print(dog[0])
```

A **dictionary** stores values using meaningful **keys**.

```
dog = {"name": "Buddy", "breed": "Labrador", "age": 4}
print(dog["name"])
```

Question: Which version makes it easier to understand what `4` represents?

Creating a Dictionary

A dictionary is made up of **key–value pairs**.

```
student = {
    "name": "Aoife",
    "student_id": 12345,
    "course": "Computing",
    "mark": 72
}
```

- Keys such as `"name"` and `"mark"` identify the information.
- Values can have different data types.
- Keys must be unique within a dictionary.

Quick check: What data type is `student["mark"]`?

Reading Values

We use a key to retrieve a value.

```
print(student["name"])
print(student["mark"])
```

We can also use the values in calculations or conditions.

```
if student["mark"] >= 40:
    print("Pass")
else:
    print("Fail")
```

Think of the key as the label on a drawer: it tells Python which value you want.

What Happens If a Key Doesn’t Exist?

This works:

```
print(student["course"])
```

But this causes a **KeyError** if `"email"` is not in the dictionary:

```
print(student["email"])
```

We can use `.get()` to provide a default:

```
print(student.get("email", "No email recorded"))
```

Question: Why might `.get()` be useful when some records have missing information?

Adding and Updating Values

Assigning to a **new key** adds information:

```
student["email"] = "aoife@example.com"
```

Assigning to an **existing key** changes the value:

```
student["mark"] = 78
```

Both use the same syntax!

```
print(student)
```

Try it: Add a `"year"` key with the value `1`.

Removing Information

Use `pop()` to remove a key and return its value.

```
dog = {
    "name": "Buddy",
    "breed": "Labrador",
    "age": 4
}

removed_age = dog.pop("age")
print(removed_age)
print(dog)
```

If the key might be missing, we can provide a default:

```
dog.pop("vaccinated", None)
```

Removing Information

Question: What would happen with `dog.pop("vaccinated")` if that key wasn't there?

Checking Whether a Key Exists

Use `in` to check **keys**.

```
dog = {"name": "Buddy", "age": 4}

if "age" in dog:
    print("We know the dog's age")

if "breed" not in dog:
    print("Breed has not been recorded")
```

Important: `"Buddy" in dog` checks whether `"Buddy"` is a **key**, not whether it is a value.

This is a common point of confusion: `in` checks dictionary keys by default.

Looping Through a Dictionary

A `for` loop can visit each key:

```
dog = {"name": "Buddy", "breed": "Labrador", "age": 4}

for key in dog:
    print(key, dog[key])
```

Or use `.items()` to get the key and value together:

```
for key, value in dog.items():
    print(key, ":", value)
```

Discuss: What does `key` contain on each iteration?

Quick Prediction Exercise

What will this print?

```
scores = {"Aoife": 70, "Ben": 35, "Cara": 62}

scores["Ben"] = 45
scores["Dara"] = 81

for name, mark in scores.items():
    if mark >= 40:
        print(name, "passed")
```

Quick Prediction Exercise

What will this print?

```
scores = {"Aoife": 70, "Ben": 35, "Cara": 62}

scores["Ben"] = 45
scores["Dara"] = 81

for name, mark in scores.items():
    if mark >= 40:
        print(name, "passed")
```

Check: Which line updates a value, and which adds a new key?

Lists of Dictionaries

A single dictionary describes one record.

A **list of dictionaries** lets us store several records:

```
dogs = [
    {"id": 1, "name": "Buddy", "age": 4},
    {"id": 2, "name": "Luna", "age": 2},
    {"id": 3, "name": "Milo", "age": 6}
]
```

We can loop through all the dogs:

```
for dog in dogs:
    print(dog["name"], dog["age"])
```

Question: Why do we use `dog["name"]` rather than `dogs["name"]`?

Searching a List of Dictionaries

Imagine we want to find the dog with ID 2.

```
for dog in dogs:
    if dog["id"] == 2:
        print("Found:", dog["name"])
```

We can put the same logic into a function:

```
def find_dog(dogs, dog_id):
    for dog in dogs:
        if dog["id"] == dog_id:
            return dog
    return None
```

Think: Why is `return None` placed after the loop?

Mini Activity: Build a Small Record

Create a dictionary describing a computer:

```
computer = {
    "device": "Laptop",
    "brand": "Dell",
    "power_on": False
}
```

1. Print the brand.
2. Add a key called `"wifi_connected"`.
3. Change `"power_on"` to `True`.
4. Use `if` to print whether the computer is powered on.
5. Use a loop to display every key and value.

Mini Activity: Build a Small Record

Try the first three tasks without looking back at the slides.